

Pseudocode: AMO Structure-Preserving Operators

(Extracted from PlatEMO A-amos implementation)

Notation: chromosome length $varDim = 2n + m - 1$; genes $1..n$ merchants, $n+1..2n$ customers; $m-1$ delimiters “0”; each merchant s must appear before customer $s+n$.

Population initialization is *outside* `OperatorAMO`: in `PlatEMO`, `Problem.Initialization()` builds the first generation (`A_amos.m` and other algorithms). This document focuses on the genetic operator module.

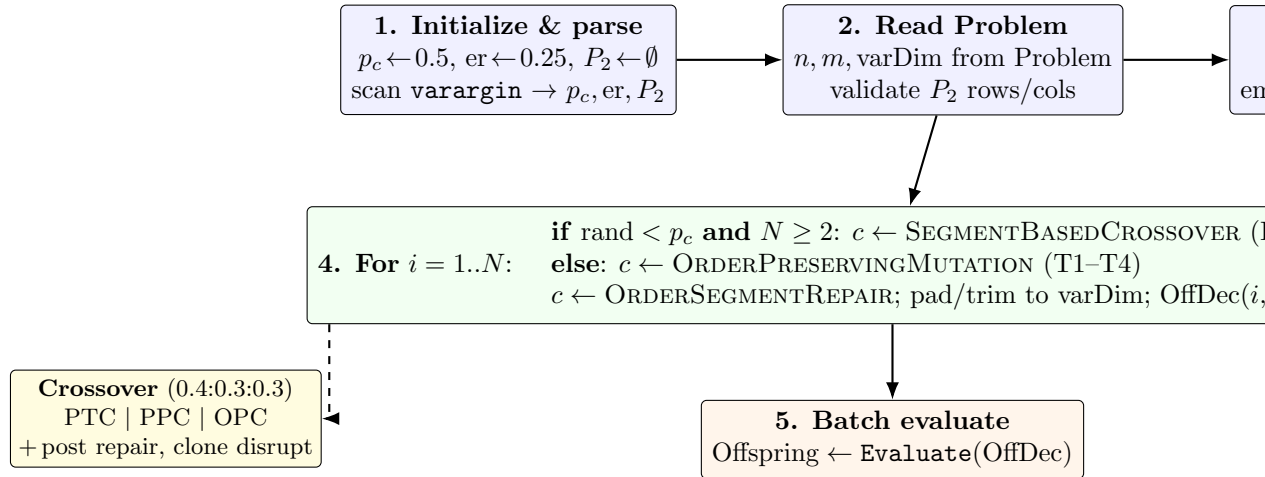


Figure 1: Overall framework of `OperatorAMO` (as implemented in `OperatorAMO.m`): parameter initialization, Problem fields, per-offspring genetic operator, unified segment repair, and batch evaluation. Side boxes summarize the three crossover modes and four mutation modes.

Algorithm 1: OperatorAMO (complete procedure, matches OperatorAMO.m)

Input: Problem, Parents (with field decs), optional key-value list
varargin

Output: Offspring = evaluated individuals

```

1  $p_c \leftarrow 0.5$ ;  $er \leftarrow 0.25$ ;  $P_2 \leftarrow \emptyset$ 
2 for  $\ell = 1$  to  $\lfloor \text{length}(\text{varargin})/2 \rfloor$  do
3    $j \leftarrow 2\ell - 1$ ;  $\text{key} \leftarrow \text{lower}(\text{varargin}(j))$ ;  $\text{val} \leftarrow \text{varargin}(j+1)$ 
4   if  $\text{key}$  is  $p_c$  then
5      $p_c \leftarrow \text{val}$ 
6   else if  $\text{key}$  is  $er$  or  $\text{explorationrate}$  then
7      $er \leftarrow \text{val}$ 
8   else if  $\text{key}$  is  $p2$  /  $\text{extraparents}$  / ... then
9      $P_2 \leftarrow \text{val}$ 
10  $n \leftarrow \text{Problem}.n$ ;  $m \leftarrow \text{Problem}.m$ ;  $\text{varDim} \leftarrow \text{Problem}.D$ 
11  $\text{ParentDec} \leftarrow \text{Parents}.decs$ 
12 if  $\text{ParentDec}$  is empty then
13   return Evaluate( $\text{ParentDec}$ ) // PlatEMO convention
14 if  $P_2 \neq \emptyset$  then
15   extract matrix  $P_2^{dec}$  from  $P_2$ 
16   if sizes of  $P_2^{dec}$  and  $\text{ParentDec}$  disagree then
17     throw size error
18 else
19    $P_2^{dec} \leftarrow []$ 
20  $N \leftarrow \text{nrows}(\text{ParentDec})$ ; allocate  $\text{OffDec} \in \mathbb{R}^{N \times \text{varDim}}$ 
21 for  $i = 1$  to  $N$  do
22    $p_1 \leftarrow \text{ParentDec}(i, :)$ 
23   if  $\text{rand}() < p_c$  and  $N \geq 2$  then
24     if  $P_2^{dec} \neq []$  then
25        $p_2 \leftarrow P_2^{dec}(i, :)$ 
26     else
27        $p_2 \leftarrow \text{draw uniformly from } \text{ParentDec}(\{1..N\} \setminus \{i\}, :)$ 
28      $c \leftarrow \text{SEGMENTBASEDCROSSOVER}(p_1, p_2, n, m, \text{varDim})$ 
29   else
30      $c \leftarrow \text{ORDERPRESERVINGMUTATION}(p_1, m, \text{varDim}, n, er)$ 
31    $c \leftarrow \text{ORDERSEGMENTREPAIR}(c, n, m, \text{varDim})$ 
32   enforce  $\text{length}(c) = \text{varDim}$  (pad with zeros or truncate)
33    $\text{OffDec}(i, :) \leftarrow c$ 
34 return Evaluate( $\text{OffDec}$ )

```

Algorithm 2: SegmentBasedCrossover

Input: $p_1, p_2, n, m, \text{varDim}$ **Output:** offspring c , strategy $\text{id} \in \{1, 2, 3\}$

```
1  $r \leftarrow \text{rand}()$ 
2 if  $r < 0.4$  then
3    $c \leftarrow \text{POSITIONTRACKINGCROSSOVER}(p_1, p_2, n, m, \text{varDim})$  // PTC
4 else if  $r < 0.7$  then
5    $c \leftarrow \text{PAIRPRIORITYCROSSOVER}(p_1, p_2, n, m, \text{varDim})$  // PPC
6 else
7    $c \leftarrow \text{ORDERPRESERVINGCROSSOVER}(p_1, p_2, n, m, \text{varDim})$  // OPC
8 trim / pad  $c$  to  $\text{varDim}$ ; remove leading/trailing zeros
9 if  $c = p_1$  or  $c = p_2$  then
10    $c \leftarrow \text{FORCEDISRUPT}(c, n, m, \text{varDim})$  // break clone
11 return  $c$ 
```

Algorithm 3: PositionTrackingCrossover (PTC)

Input: valid parents p_1, p_2 **Output:** offspring c

```
1 build position maps  $\pi_1, \pi_2$  for genes  $1..2n$ 
2  $\mathcal{K} \leftarrow \{k \in [1, \text{varDim}-1] : k \text{ is safe for both } \pi_1, \pi_2\}$ 
   // safe: no pair  $(s, s+n)$  with  $\pi(s) < k < \pi(s+n)$ 
3 if  $\mathcal{K} = \emptyset$  then
4   return  $p_1$ 
5 sample  $k \in \mathcal{K}$  uniformly
6  $c(1 : k) \leftarrow p_1(1 : k)$ ;  $U \leftarrow$  multiset of genes in  $c(1 : k)$ 
7  $i \leftarrow k+1$ 
8 for scan index  $t$  from 1 to  $\text{varDim}$  do
9    $g \leftarrow p_2(t)$ 
10  if  $g \notin U$  then
11    if  $g = 0$  then
12      append  $g$ ; add to  $U$ 
13    else if  $n < g \leq 2n$  then
14      if  $\text{merchant}(g-n) \in U$  then
15        append  $g$ ; add to  $U$ 
16    else
17      append  $g$ ; add to  $U$ 
18  if filled up to  $\text{varDim}$  then
19    break
20  $c \leftarrow \text{REPAIRSOLUTION}(c, n, m, \text{varDim})$ 
21 if not  $\text{ISVALID}(c)$  then
22    $c \leftarrow \text{REPAIRORDER}(c, n)$ 
23  $c \leftarrow \text{REPAIRSEGMENTS}(c, n, m)$ 
24 return  $c$ 
```

Algorithm 4: PairPriorityCrossover (PPC)

Input: valid parents p_1, p_2

Output: offspring c

```
1 for  $s = 1$  to  $n$  do
2    $\text{pri}_1(s) \leftarrow$  first index of  $s$  in  $p_1$ 
3    $\text{pri}_2(s) \leftarrow$  first index of  $s$  in  $p_2$ 
4    $\text{merged}(s) \leftarrow \min(\text{pri}_1(s), \text{pri}_2(s))$ 
5 sort merchants by increasing  $\text{merged}(s) \rightarrow$  list  $M$ 
6 initialize  $c \leftarrow \text{zeros}(1, \text{varDim})$ ; write  $M(1), M(2), \dots$  into  $c$  from the left
7 foreach merchant  $s$  in  $M$  do
8   find position  $\ell$  of  $s$  in  $c$ 
9   scan right from  $\ell+1$ : set first slot still 0 to customer  $s+n$ 
10  $c \leftarrow \text{REPAIRSOLUTION}(c, n, m, \text{varDim})$ 
11 if not  $\text{ISVALID}(c)$  then
12    $c \leftarrow \text{REPAIRORDER}(c, n)$ 
13  $c \leftarrow \text{REPAIRSEGMENTS}(c, n, m)$ 
14 return  $c$ 
```

Algorithm 5: OrderPreservingCrossover (OPC)

Input: valid parents p_1, p_2

Output: offspring c

```
1 extract merchant visit orders  $O_1, O_2$  (first occurrences, duplicates removed)
2 if  $O_1$  or  $O_2$  empty then
3   return  $p_1$ 
4  $f \leftarrow \text{randint}(1, |O_1|)$ 
5  $F \leftarrow \text{unique}([O_1(1 : f), O_2(f+1 : \text{end})], \text{stable})$ 
6 append missing merchants from  $O_1 \cup O_2$  to  $F$ 
7 if  $F = O_1$  and  $|F| \geq 2$  then
8   swap two random positions in  $F$ 
9 build  $c$  from  $F$  exactly as in PPC (write merchants, then fill first zero after each)
10  $c \leftarrow \text{REPAIRSOLUTION}(c, n, m, \text{varDim})$ 
11 if not  $\text{ISVALID}(c)$  then
12    $c \leftarrow \text{REPAIRORDER}(c, n)$ 
13  $c \leftarrow \text{REPAIRSEGMENTS}(c, n, m)$ 
14 return  $c$ 
```

Algorithm 6: OrderPreservingMutation

Input: individual x , counts m, n , dimension varDim , exploration rate ρ
Output: mutated x'

```
1 split  $x$  into segments  $S_1..S_m$  by  $m-1$  zeros
2 if  $\text{rand}() < \rho$  then
3   |  $x' \leftarrow \text{TYPE4\_SEGSWAP}(x, \{S_k\}, m, n, \text{varDim})$  // large step
4 else
5   |  $t \leftarrow \text{randint}(1, 3)$ 
6   | if  $t = 1$  then
7     |  $x' \leftarrow \text{TYPE1}(x)$  // random merchant to segment head
8   | else if  $t = 2$  then
9     |  $x' \leftarrow \text{TYPE2}(x)$  // random customer to segment tail
10  | else
11    |  $x' \leftarrow \text{TYPE3\_MOVEDELIMITER}(x)$  // slide delimiter ‘‘0’’
12 fix length and zero count; if  $x' = x$  then
13   | retry Type4 / intra-segment pair swap
14 return  $x'$ 
```

Algorithm 7: Mutation Type1: move random merchant to *head* of its segment

Input: individual x , segments $\{S_k\}$, delimiter index array E (extended zero positions)
Output: x'

```
1  $\mathcal{V} \leftarrow \{k : S_k \text{ contains some merchant in } \{1..n\}\}$ 
2 if  $\mathcal{V} = \emptyset$  then
3   | return  $x$ 
4 sample  $k \in \mathcal{V}$ ; let  $\sigma \leftarrow S_k$ 
5 pick random merchant gene  $a$  from  $\sigma$ 
6 remove  $a$  from its position in  $\sigma$ ;  $\sigma' \leftarrow [a, \sigma]$ 
7  $x' \leftarrow x$ ; write  $\sigma'$  back into positions  $(E(k)+1)..(E(k+1)-1)$ 
8 return  $x'$ 
```

Algorithm 8: Mutation Type2: move random customer to *tail* of its segment

Input: individual x , segments $\{S_k\}$, extended indices E
Output: x'

```
1  $\mathcal{V} \leftarrow \{k : S_k \text{ contains some customer in } \{n+1..2n\}\}$ 
2 if  $\mathcal{V} = \emptyset$  then
3   | return  $x$ 
4 sample  $k \in \mathcal{V}$ ;  $\sigma \leftarrow S_k$ 
5 pick random customer  $b$  from  $\sigma$ 
6 remove  $b$ ;  $\sigma' \leftarrow [\sigma, b]$ 
7  $x' \leftarrow x$ ; write  $\sigma'$  into segment  $k$ 
8 return  $x'$ 
```

Algorithm 9: Mutation Type3: slide one delimiter “0” (pair-safe boundaries)

Input: x , segments $\{S_k\}$, E , counts m, n , varDim
Output: x'

```

1 if  $m \leq 1$  then
2   return  $x$ 
3 pick movable delimiter index  $z \in \{1..m-1\}$  (adjacent segments
   nonempty)
4  $d \leftarrow \text{randint}(0, 1)$  // 0: scan backward in left segment; 1:
   forward in right
5 if  $d = 0$  then
6    $q \leftarrow \text{TRAVERSEFORWARD}(x, E, z, S_z, n)$  // first valid slot
   before a merchant
7 else
8    $q \leftarrow \text{TRAVERSEBACKWARD}(x, E, z, S_{z+1}, n)$  // first valid slot
   after a customer
9 if  $q < 0$  then
10  return  $x$ 
11 remove the old “0” at global index  $E(z+1)$  from  $x$ ; adjust  $q$  if needed
12 insert “0” at global index  $q$ ; pad/trim to varDim
13 return  $x'$ 

```

Algorithm 10: TraverseForward / TraverseBackward (Type3 helpers, sketch)

Input: segment vector, delimiter context, n
Output: global index for new “0”, or -1 if impossible

```

1 TRAVERSEFORWARD scans the left segment from tail to head
2   at each merchant  $a$ , let  $C$  be customers in the already-scanned suffix
3   if  $C$  is empty or  $C \subseteq (\text{merchants scanned} \cup \{a\}) + n$  and slot before
    $a$  is legal then return that index
4 TRAVERSEBACKWARD scans the right segment from head to tail
5   at each customer  $c$ , let  $M$  be merchants in the already-scanned
   prefix
6   if  $M$  is empty or  $M \subseteq (\text{customers scanned} \cup \{c\}) - n$  and slot after
    $c$  is legal then return that index
7 return  $-1$ 

```

Algorithm 11: Type4_SegSwap (sketch)

Input: segments $\{S_k\}$

- 1 choose source k_s with at least two merchant–customer pairs
- 2 pick random merchant a in S_{k_s} with both a and $a+n$ present
- 3 remove $(a, a+n)$ from S_{k_s} preserving order
- 4 choose target $k_t \neq k_s$; random insertion position in S_{k_t}
- 5 insert consecutive $(a, a+n)$
- 6 rebuild chromosome; **if** *constraints violated* **then**
- 7 **return** x unchanged
- 8 **return** rebuilt chromosome

Algorithm 12: Repair pipeline (abstract)

Input: chromosome c

- 1 ENSURENOZEROATENDS(c)
- 2 REPAIRSEGMENTS(c) // move customers to merchant segments
- 3 REPAIRSEGMENTSIZES(c, m) // each segment has even length ≥ 2
- 4 pad/trim to varDim
- 5 **return** c
